

Rain OS Plugin — Backend Upgrade Sync

Context

The Rain OS backend has been upgraded from 3 pillars to 4 pillars. The WordPress plugin was built against the old 3-pillar backend. This prompt details every change needed to bring the plugin fully in line with the current backend response shape. Do not change any styling or UX unless explicitly stated. Only fix the data wiring.

What the backend now returns

Every `POST /api/analyze` response now includes:

```
```json
{
 "success": true,
 "overall_score": 78,

 "ai_readability": 82,
 "digital_authority": 75,
 "conversion_readiness": 71,
 "product_discoverability": 84,

 "pillars": {
 "ai_readability": 82,
 "digital_authority": 75,
 "conversion_readiness": 71,
 "product_discoverability": 84
 },

 "ai_scores": {
 "readability": 85,
 "structure": 78,
 "freshness": 72,
 "citation": 81,
 "visibility": 76
 },

 "sub_scores": {
 "semanticClarity": 85,
 "readabilityScore": 78,
 "logicalStructure": 80,
```

```
"aeoAlignment": 74,
"entityRecognition": 72,
"citationReadiness": 81,
"descriptiveMetadata": 77,
"schemaExtraction": 68,
"qaFormatDetection": 73,
"metadataAudit": 69
},
```

```
"phase2_sub_scores": {
 "content_chunking_quality": 80,
 "information_gain_score": 74,
 "js_dependency_flag": 100,
 "entity_consistency": 77,
 "meta_description_aeo_quality": 65,
 "product_content_connection": 71
},
```

```
"crawler_signals": {
 "ai_crawler_access": "open",
 "ai_crawler_access_score": 100,
 "llms_txt_status": "missing",
 "llms_txt_permissions": 20,
 "crawler_conversion_risk": "medium",
 "crawler_conversion_score": 40,
 "schema_completeness": 55,
 "schema_maturity_level": "basic",
 "p4_llms_txt_status": "missing",
 "p4_llms_txt_permissions": 20,
 "p4_ai_crawler_permissions": "open",
 "p4_ai_crawler_access_score": 100
},
```

```
"ai_readability_detail": {
 "semantic_clarity": 85,
 "readability_score": 78,
 "logical_structure": 80,
 "aeo_alignment": 74,
 "ai_crawler_access": "open",
 "ai_crawler_access_score": 100
},
```

```
"digital_authority_detail": {
 "entity_recognition": 72,
```

```
"citation_readiness": 81,
"descriptive_metadata": 77,
"llms_txt_status": "missing",
"llms_txt_permissions": 20
},

"conversion_readiness_detail": {
 "schema_extraction": 68,
 "qa_format_detection": 73,
 "metadata_audit": 69,
 "crawler_conversion_risk": "medium",
 "crawler_conversion_score": 40
},

"product_discoverability_detail": {
 "schema_completeness": 55,
 "schema_maturity_level": "basic",
 "answer_layer_quality": 80,
 "direct_answer_readiness": 77,
 "qa_format_density": 73,
 "entity_disambiguation": 76,
 "semantic_quality_score": 82,
 "conversational_query_match": 79,
 "attribute_completeness": 60,
 "price_availability_clarity": 45,
 "freshness_signals": 72,
 "structured_content_quality": 80,
 "llms_txt_status": "missing",
 "llms_txt_permissions": 20,
 "ai_crawler_permissions": "open",
 "ai_crawler_access_score": 100
},

"recommendations": [
 {
 "title": "Add LLMs.txt file",
 "description": "Create an LLMs.txt file at your domain root to explicitly grant AI citation rights.
Without it, AI systems treat your site as unverified.",
 "icon": "crawl",
 "color": "cyan",
 "pillar": "digital_authority",
 "priority": 1
 }
],
```

```

"authorship": {
 "hash": "A1B2C3D4E5F6",
 "timestamp": "2026-02-23T10:00:00.000Z",
 "status": "Pending"
},

"usage": {
 "count": 5,
 "limit": 100
}
}
...

```

---

## Changes required — file by file

---

### 1. `includes/api/class-rairo-api-client.php`

**\*\*Change 1 — `analyze\_content()` must accept and forward `\$content\_id`\*\***

Replace:

```

```php
public function analyze_content( $content, $industry = " ) {
    return $this->make_request(
        'analyze',
        'POST',
        array(
            'action' => 'full_analysis',
            'content' => $content,
            'industry' => $industry,
        )
    );
}
...

```

With:

```

```php
public function analyze_content($content, $industry = "", $content_id = ") {
 $payload = array(
 'action' => 'full_analysis',

```

```

 'content' => $content,
 'industry' => $industry,
);
 if (! empty($content_id)) {
 $payload['contentId'] = $content_id;
 }
 return $this->make_request('analyze', 'POST', $payload);
}
...

```

**\*\*Change 2 — `parse\_analysis\_response()` must pass through all new fields\*\***

Replace the entire `parse\_analysis\_response()` method with:

```

```php
public function parse_analysis_response( $response ) {
    if ( is_wp_error( $response ) ) {
        return $response;
    }

    // Support both camelCase (raw Gemini shape) and snake_case (normalized shape)
    $overall    = isset( $response['overall_score'] ) ? intval( $response['overall_score'] ) : (
isset( $response['overallScore'] ) ? intval( $response['overallScore'] ) : 0 );
    $ai_read    = isset( $response['ai_readability'] ) ? intval( $response['ai_readability'] ) : (
isset( $response['pillarScores']['aiReadability'] ) ? intval( $response['pillarScores']['aiReadability']
) : 0 );
    $dig_auth   = isset( $response['digital_authority'] ) ? intval( $response['digital_authority'] ) : (
isset( $response['pillarScores']['digitalAuthority'] ) ? intval(
$response['pillarScores']['digitalAuthority'] ) : 0 );
    $conv_read  = isset( $response['conversion_readiness'] ) ? intval(
$response['conversion_readiness'] ) : ( isset( $response['pillarScores']['conversionReadiness'] )
? intval( $response['pillarScores']['conversionReadiness'] ) : 0 );
    $prod_disc  = isset( $response['product_discoverability'] ) ? intval(
$response['product_discoverability'] ) : ( isset( $response['pillarScores']['productDiscoverability']
) ? intval( $response['pillarScores']['productDiscoverability'] ) : 0 );

    $parsed = array(
        'overall_score'      => $overall,
        'ai_readability'     => $ai_read,
        'digital_authority'  => $dig_auth,
        'conversion_readiness' => $conv_read,
        'product_discoverability' => $prod_disc,

        'pillars' => array(
            'ai_readability'    => $ai_read,

```

```

        'digital_authority'    => $dig_auth,
        'conversion_readiness' => $conv_read,
        'product_discoverability' => $prod_disc,
    ),

    'ai_scores'                => isset( $response['ai_scores'] )      ? $response['ai_scores']
: array(),
    'sub_scores'               => array(),
    'phase2_sub_scores'        => isset( $response['phase2_sub_scores'] ) ?
$response['phase2_sub_scores'] : array(),
    'crawler_signals'          => isset( $response['crawler_signals'] ) ?
$response['crawler_signals'] : array(),

    'ai_readability_detail'    => isset( $response['ai_readability_detail'] )      ?
$response['ai_readability_detail'] : array(),
    'digital_authority_detail' => isset( $response['digital_authority_detail'] )      ?
$response['digital_authority_detail'] : array(),
    'conversion_readiness_detail' => isset( $response['conversion_readiness_detail'] ) ?
$response['conversion_readiness_detail'] : array(),
    'product_discoverability_detail' => isset( $response['product_discoverability_detail'] ) ?
$response['product_discoverability_detail'] : array(),

    'recommendations'          => isset( $response['recommendations'] )      ?
$response['recommendations'] : array(),
    'keywords'                  => isset( $response['keywords'] )              ? $response['keywords']
: array(),
    'authorship'                => isset( $response['authorship'] )            ? $response['authorship']
: null,
);

// sub_scores: support both array-of-objects and flat key=>value
if ( isset( $response['sub_scores'] ) ) {
    if ( is_array( $response['sub_scores'] ) ) {
        foreach ( $response['sub_scores'] as $key => $val ) {
            if ( is_array( $val ) && isset( $val['category'] ) ) {
                $parsed['sub_scores'][$val['category']] = intval( $val['score'] );
            } else {
                $parsed['sub_scores'][$key] = intval( $val );
            }
        }
    }
}
} elseif ( isset( $response['subScores'] ) && is_array( $response['subScores'] ) ) {
    foreach ( $response['subScores'] as $sub ) {
        if ( isset( $sub['category'], $sub['score'] ) ) {

```

```

        $parsed['sub_scores'][$sub['category']] = intval( $sub['score'] );
    }
}
}

return $parsed;
}
...

```

2. `includes/class-rairo-ajax.php`

****Change 1 — `analyze\_content()` must build and send `content\_id`****

Find the `analyze\_content()` method. After the line that reads `\$post\_id = ...`, add:

```

```php
$content_id = $post_id > 0 ? 'wp_post_' . $post_id : '';
...

```

Then change the `analyze\_content` API call from:

```

```php
$result = $this->api_client->analyze_content( $full_content, $industry );
...

```

To:

```

```php
$result = $this->api_client->analyze_content($full_content, $industry, $content_id);
...

```

**\*\*Change 2 — `save\_analysis\_to\_history()` must save Pillar 4\*\***

Replace the entire `save\_analysis\_to\_history()` method with:

```

```php
private function save_analysis_to_history( $post_id, $analysis ) {
    global $wpdb;
    $table_name = $wpdb->prefix . 'rairo_analysis_history';

    $wpdb->insert(
        $table_name,
        array(
            'post_id'          => $post_id,
            'overall_score'    => isset( $analysis['overall_score'] ) ? absint(
                $analysis['overall_score'] ) : 0,

```

```

        'ai_readability'      => isset( $analysis['ai_readability'] )      ? absint(
$analysis['ai_readability'] )      : 0,
        'digital_authority'   => isset( $analysis['digital_authority'] )   ? absint(
$analysis['digital_authority'] )   : 0,
        'conversion_readiness' => isset( $analysis['conversion_readiness'] ) ? absint(
$analysis['conversion_readiness'] ) : 0,
        'product_discoverability' => isset( $analysis['product_discoverability'] ) ? absint(
$analysis['product_discoverability'] ) : 0,
        'analysis_data'       => wp_json_encode( $analysis ),
        'analyzed_at'         => current_time( 'mysql' ),
    ),
    array( '%d', '%d', '%d', '%d', '%d', '%d', '%s', '%s' )
);
}
...

```

****Change 3 — `get\_dashboard\_data()` must include Pillar 4 in averages****

In the SQL query inside `get\_dashboard\_data()`, add `ROUND(AVG(product\_discoverability))` as `avg\_product\_discoverability` to the SELECT list so it reads:

```

```sql
SELECT
 ROUND(AVG(overall_score)) as avg_overall,
 ROUND(AVG(ai_readability)) as avg_ai_readability,
 ROUND(AVG(digital_authority)) as avg_digital_authority,
 ROUND(AVG(conversion_readiness)) as avg_conversion_readiness,
 ROUND(AVG(product_discoverability)) as avg_product_discoverability,
 COUNT(*) as total_analyzed
FROM {$table_name}
WHERE analyzed_at >= %s
```

```

Also update the trend_data query to include:

```

```sql
ROUND(AVG(product_discoverability)) as avg_product_discoverability
```

```

****Change 4 — `analyze\_content()` response must include all fields for admin.js****

Change the final `wp\_send\_json\_success()` call in `analyze\_content()` from:

```

```php
wp_send_json_success(array(
 'analysis' => $parsed,
 'usage' => $usage_info,

```



```
);
...
```

To:

```
```php  
wp_send_json_success( array_merge( $parsed, array( 'usage' => $usage_info ) ) );  
...
```

This makes `response.data.pillars`, `response.data.ai\_scores`, `response.data.crawler\_signals` etc. directly accessible to admin.js without it needing to dig into a nested `analysis` key.

3. `assets/js/admin.js`

****Change 1 — `displayResults()` must render Pillar 4****

Find the block inside `displayResults()` that renders pillars:

```
```javascript  
html += this.renderPillarScore('AI Readability', data.pillars.ai_readability || 0, 'cyan');
html += this.renderPillarScore('Digital Authority', data.pillars.digital_authority || 0, 'green');
html += this.renderPillarScore('Conversion Readiness', data.pillars.conversion_readiness || 0,
'purple');
...
```

Add a fourth line immediately after:

```
```javascript  
html += this.renderPillarScore('Product Discoverability', data.pillars.product_discoverability || 0,  
'yellow');  
...
```

****Change 2 — `displayRecommendations()` must handle rich recommendation objects****

The backend now returns recommendations as objects `{title, description, icon, color, pillar, priority}` not plain strings. Find `displayRecommendations()` and replace the loop body so it handles both shapes:

```
```javascript  
for (var i = 0; i < recommendations.length; i++) {
 var rec = recommendations[i];
 if (typeof rec === 'string') {
 html += '<div class="rain-os-rec-item"><p>' + rec + '</p></div>';
 } else {
 var priorityClass = rec.priority === 1 ? 'critical' : rec.priority === 2 ? 'important' :
'enhancement';
 html += '<div class="rain-os-rec-item rain-os-rec-' + priorityClass + '">';
 html += '<div class="rain-os-rec-header">';
```

```

 html += '' + rec.title + '';
 html += '' + (rec.pillar || "").replace(/_/g, ' ') + '';
 html += '</div>';
 html += '<p class="rain-os-rec-description">' + rec.description + '</p>';
 html += '</div>';
 }
}
...

```

**\*\*Change 3 — `displayAIScores()` must read from `data.ai\_scores` not `data.scores`\*\***

Find the line inside the analyze success callback that calls `displayAIScores`. It currently reads:

```

```javascript
if (response.data.ai_scores) {
  self.displayAIScores(response.data.ai_scores);
}
...

```

This is already correct. But also find any place that calls
`self.displayAIScores(response.data.scores)` (without the `ai\_` prefix) and change it to
`response.data.ai\_scores`.

****Change 4 — add `displayCrawlerSignals()` function****

After `displayAIScores`, add this new function:

```

```javascript
displayCrawlerSignals: function(signals) {
 if (!signals) return;
 var $container = $('#rain-os-crawler-signals');
 if (!$container.length) return;

 var statusLabel = function(val) {
 var map = { open: 'Open', restricted: 'Restricted', blocked: 'Blocked', unknown: 'Unknown',
 present: 'Present', missing: 'Missing', partial: 'Partial', blocking: 'Blocking',
 none: 'None', low: 'Low', medium: 'Medium', high: 'High' };
 return map[val] || val;
 };

 var html = '<div class="rain-os-crawler-grid">';
 html += '<div class="rain-os-crawler-item">AI Crawler Access' +
statusLabel(signals.ai_crawler_access) + ' (' + (signals.ai_crawler_access_score || 0) +
')</div>';
 html += '<div class="rain-os-crawler-item">LLMs.txt Status' +
statusLabel(signals.llms_txt_status) + ' (' + (signals.llms_txt_permissions || 0) +
')</div>';

```

```

 html += '<div class="rain-os-crawler-item">Conversion Risk' +
statusLabel(signals.crawler_conversion_risk) + ' (' + (signals.crawler_conversion_score || 0) +
')</div>';
 html += '<div class="rain-os-crawler-item">Schema Maturity' +
statusLabel(signals.schema_maturity_level) + ' (' + (signals.schema_completeness || 0) +
')</div>';
 html += '</div>';

 $container.html(html).show();
},
...

```

Then in the analyze success callback, after calling `displayAIScores`, add:

```

```javascript
if (response.data.crawler_signals) {
    self.displayCrawlerSignals(response.data.crawler_signals);
}
...

---

```

4. `templates/pillar-breakdown.php`

This file has three critical problems:

****Problem 1 — Wrong table name****

Line 8 uses `\$wpdb->prefix . 'ai\_readability\_analysis\_history'`.
Change it to: `\$wpdb->prefix . 'rairo\_analysis\_history'`

****Problem 2 — Missing Pillar 4 in SQL and display****

Replace the entire SQL query and PHP variable block at the top with:

```

```php
$averages = $wpdb->get_row(
 $wpdb->prepare(
 "SELECT
 ROUND(AVG(ai_readability)) as avg_ai_readability,
 ROUND(AVG(digital_authority)) as avg_digital_authority,
 ROUND(AVG(conversion_readiness)) as avg_conversion_readiness,
 ROUND(AVG(product_discoverability)) as avg_product_discoverability,
 COUNT(*) as total_analyzed
 FROM {$table_name}
 WHERE analyzed_at >= %s",

```

```

 $date_limit
),
ARRAY_A
);

$ai_readability = isset($averages['avg_ai_readability']) ? intval(
$averages['avg_ai_readability']) : 0;
$digital_authority = isset($averages['avg_digital_authority']) ? intval(
$averages['avg_digital_authority']) : 0;
$conversion_readiness = isset($averages['avg_conversion_readiness']) ? intval(
$averages['avg_conversion_readiness']) : 0;
$product_discoverability = isset($averages['avg_product_discoverability']) ? intval(
$averages['avg_product_discoverability']) : 0;
$overall_score = $ai_readability + $digital_authority + $conversion_readiness +
$product_discoverability > 0
 ? round(($ai_readability + $digital_authority + $conversion_readiness +
$product_discoverability) / 4)
 : 0;
...

```

### **\*\*Problem 3 — Fake sub-scores\*\***

The current code fabricates sub-scores by adding/subtracting from pillar averages (e.g. ` \$ai\_semantic = \$ai\_readability + 3 `). Replace all of this with a query that reads real sub-score data from the most recent `analysis\_data` JSONB column:

```

```php
// Read real sub-scores from most recent analysis row
$latest = $wpdb->get_var( "SELECT analysis_data FROM {$table_name} ORDER BY
analyzed_at DESC LIMIT 1" );
$latest_data = $latest ? json_decode( $latest, true ) : array();

$p1 = isset( $latest_data['ai_readability_detail'] )      ? $latest_data['ai_readability_detail']
: array();
$p2 = isset( $latest_data['digital_authority_detail'] )    ? $latest_data['digital_authority_detail']
: array();
$p3 = isset( $latest_data['conversion_readiness_detail'] ) ?
$latest_data['conversion_readiness_detail'] : array();
$p4 = isset( $latest_data['product_discoverability_detail'] ) ?
$latest_data['product_discoverability_detail'] : array();
$crawler = isset( $latest_data['crawler_signals'] ) ? $latest_data['crawler_signals'] : array();

// Helper to safely read a sub-score
function rairo_sub( $arr, $key, $fallback = 0 ) {

```

```

    return isset( $arr[ $key ] ) ? intval( $arr[ $key ] ) : $fallback;
}
...

```

Then replace the hardcoded sub-score variables with:

```

```php
$ai_semantic = rairo_sub($p1, 'semantic_clarity', $ai_readability);
$ai_read_score = rairo_sub($p1, 'readability_score', $ai_readability);
$ai_structure = rairo_sub($p1, 'logical_structure', $ai_readability);
$ai_aeo = rairo_sub($p1, 'aeo_alignment', $ai_readability);

$da_entity = rairo_sub($p2, 'entity_recognition', $digital_authority);
$da_citation = rairo_sub($p2, 'citation_readiness', $digital_authority);
$da_metadata = rairo_sub($p2, 'descriptive_metadata', $digital_authority);

$cr_schema = rairo_sub($p3, 'schema_extraction', $conversion_readiness);
$cr_qa = rairo_sub($p3, 'qa_format_detection', $conversion_readiness);
$cr_metadata = rairo_sub($p3, 'metadata_audit', $conversion_readiness);

$pd_schema = rairo_sub($p4, 'schema_completeness', $product_discoverability);
$pd_answer = rairo_sub($p4, 'answer_layer_quality', $product_discoverability);
$pd_entity = rairo_sub($p4, 'entity_disambiguation', $product_discoverability);
$pd_query = rairo_sub($p4, 'conversational_query_match', $product_discoverability);
$pd_freshness = rairo_sub($p4, 'freshness_signals', $product_discoverability);
$pd_structured = rairo_sub($p4, 'structured_content_quality', $product_discoverability);
...

```

**\*\*Problem 4 — Add Pillar 4 section to the HTML\*\***

After the closing `

` of the Conversion Readiness pillar section, add a fourth pillar card:

```

```html
<div class="rain-os-pillar-section rain-os-pillar-yellow">
  <h3 class="rain-os-pillar-title"><?php esc_html_e( 'Product Discoverability',
'rain-os-ai-readability-optimizer' ); ?></h3>
  <div class="rain-os-pillar-score"><?php echo esc_html( $product_discoverability ); ?>%</div>

  <div class="rain-os-subcategory">
    <div class="rain-os-subcategory-header">
      <span><?php esc_html_e( 'Schema Completeness', 'rain-os-ai-readability-optimizer' );
?></span>
      <span><?php echo esc_html( $pd_schema ); ?>%</span>
    </div>
    <div class="rain-os-bar-track">

```

```

        <div class="rain-os-bar-fill rain-os-bar-yellow-1" style="width: <?php echo esc_attr(
$pd_schema ); ?>%;"></div>
    </div>
</div>

<div class="rain-os-subcategory">
    <div class="rain-os-subcategory-header">
        <span><?php esc_html_e( 'Answer Layer Quality', 'rain-os-ai-readability-optimizer' );
?></span>
        <span><?php echo esc_html( $pd_answer ); ?>%</span>
    </div>
    <div class="rain-os-bar-track">
        <div class="rain-os-bar-fill rain-os-bar-yellow-2" style="width: <?php echo esc_attr(
$pd_answer ); ?>%;"></div>
    </div>
</div>

<div class="rain-os-subcategory">
    <div class="rain-os-subcategory-header">
        <span><?php esc_html_e( 'Freshness Signals', 'rain-os-ai-readability-optimizer' );
?></span>
        <span><?php echo esc_html( $pd_freshness ); ?>%</span>
    </div>
    <div class="rain-os-bar-track">
        <div class="rain-os-bar-fill rain-os-bar-yellow-3" style="width: <?php echo esc_attr(
$pd_freshness ); ?>%;"></div>
    </div>
</div>

<div class="rain-os-subcategory">
    <div class="rain-os-subcategory-header">
        <span><?php esc_html_e( 'Conversational Query Match',
'rain-os-ai-readability-optimizer' ); ?></span>
        <span><?php echo esc_html( $pd_query ); ?>%</span>
    </div>
    <div class="rain-os-bar-track">
        <div class="rain-os-bar-fill rain-os-bar-yellow-4" style="width: <?php echo esc_attr(
$pd_query ); ?>%;"></div>
    </div>
</div>

<!-- Crawler signals section for Pillar 4 -->
<?php if ( ! empty( $crawler ) ) : ?>
<div class="rain-os-crawler-signals">

```

```

        <h4><?php esc_html_e( 'Crawler Signals', 'rain-os-ai-readability-optimizer' ); ?></h4>
        <div class="rain-os-crawler-row">
            <span><?php esc_html_e( 'AI Crawler Access', 'rain-os-ai-readability-optimizer' );
?></span>
            <strong><?php echo esc_html( ucfirst( $crawler['ai_crawler_access'] ?? 'unknown' ) );
?> (<?php echo esc_html( $crawler['ai_crawler_access_score'] ?? 0 ); ?>)</strong>
        </div>
        <div class="rain-os-crawler-row">
            <span><?php esc_html_e( 'LLMs.txt Status', 'rain-os-ai-readability-optimizer' );
?></span>
            <strong><?php echo esc_html( ucfirst( $crawler['llms_txt_status'] ?? 'missing' ) ); ?>
(<?php echo esc_html( $crawler['llms_txt_permissions'] ?? 0 ); ?>)</strong>
        </div>
        <div class="rain-os-crawler-row">
            <span><?php esc_html_e( 'Schema Maturity', 'rain-os-ai-readability-optimizer' );
?></span>
            <strong><?php echo esc_html( ucfirst( $crawler['schema_maturity_level'] ?? 'none' ) );
?></strong>
        </div>
    </div>
    <?php endif; ?>
</div>
...

```

5. `templates/dashboard.php`

****Change 1 — SQL must include `product\_discoverability`****

In the averages query, add:

```

```sql
ROUND(AVG(product_discoverability)) as avg_product_discoverability
...

```

**\*\*Change 2 — PHP variable\*\***

After the existing pillar variable assignments, add:

```

```php
$product_discoverability = isset( $averages['avg_product_discoverability'] ) ? intval(
$averages['avg_product_discoverability'] ) : 0;
...

```

And update the `\$overall\_score` calculation from dividing by 3 to dividing by 4:

```

```php
$overall_score = round(($ai_readability + $digital_authority + $conversion_readiness +
$product_discoverability) / 4);
```

```

****Change 3 — HTML must show Pillar 4 bar****

After the Conversion Readiness bar block, add:

```

```html
<div class="rain-os-pillar-bar">
 <div class="rain-os-pillar-bar-label">
 <?php esc_html_e('Product Discoverability', 'rain-os-ai-readability-optimizer');
?>
 <?php echo esc_html($product_discoverability); ?>%
 </div>
 <div class="rain-os-pillar-bar-track">
 <div class="rain-os-pillar-bar-fill rain-os-pillar-yellow" style="width: <?php echo esc_attr(
$product_discoverability); ?>%;"></div>
 </div>
</div>
```

```

6. `templates/score-history.php`

****Change 1 — Average calculation must use 4 pillars****

Find:

```

```php
$avg_score = round((intval($item['ai_readability']) + intval($item['digital_authority']) + intval(
$item['conversion_readiness'])) / 3);
```

```

Replace with:

```

```php
$pd = isset($item['product_discoverability']) ? intval($item['product_discoverability']) : 0;
$avg_score = round((intval($item['ai_readability']) + intval($item['digital_authority']) + intval(
$item['conversion_readiness']) + $pd) / 4);
```

```

****Change 2 — Add Pillar 4 column to the table****

In the table header row, after the Conversion Readiness ` >`, add: |

```

```html

```



```
<th><?php esc_html_e('Product Discoverability', 'rain-os-ai-readability-optimizer'); ?></th>
...
```

In the table data rows, after the Conversion Readiness ` ` , add: |

```
```html
<td>
    <span class="rain-os-score-indicator rain-os-score-<?php echo esc_attr(
ai_readability_score_class( isset( $item['product_discoverability'] ) ? intval(
$item['product_discoverability'] ) : 0 ) ); ?>"></span>
    <span class="rain-os-score-value"><?php echo esc_html( isset(
$item['product_discoverability'] ) ? $item['product_discoverability'] : '—' ); ?></span>
</td>
...
---
```

7. `assets/js/ai-score-panel.js`

No structural changes needed. The panel already reads `response.data.scores.readability` etc. and the backend already returns `ai\_scores` with those exact keys. Confirm the AJAX call sends `content\_id` as `wp\_post\_` + `postId` — if it doesn't, add it to the data payload:

```
```javascript
data: {
 action: 'rairo_get_ai_scores',
 nonce: rainOsAeoPanelData.nonce,
 content_id: 'wp_post_' + rainOsAeoPanelData.postId
}
...

```

## Summary of what is broken without these changes

Area	Current behaviour	After fix
Pillar 4 score	Never shown anywhere	Shown in analyzer, dashboard, history, breakdown
Pillar Breakdown page	Wrong table name = blank page	Reads real data
Sub-scores on breakdown page	Fake numbers (pillar ± random offset)	Real Gemini-scored values
Recommendations	Crashes if object format returned	Renders title + description + pillar
Dashboard overall score	Divides 3 pillars by 3	Divides 4 pillars by 4 (correct)
Score history average	Divides by 3	Divides by 4
Score panel (post editor)	Never finds row (content_id not sent)	Populates correctly after first analysis

| Crawler signals | Never displayed | Shown in analyzer and pillar breakdown |

---

## How scores are calculated — essential context for debugging

This section explains the scoring engine so you understand what the plugin must send and why. Do not change any of this logic — it lives in the backend. But you need to understand it to wire the plugin correctly.

---

### The two-stage scoring pipeline

Every analysis runs through two stages before a score is returned.

**\*\*Stage 1 — Local algorithmic measurement (no AI, pure maths)\*\***

Before Gemini sees anything, the backend runs the raw content through a local algorithm (`readability.ts`) that measures 15 specific things directly from the text:

| Metric | What it measures |

|---|---|

| Passive voice ratio | Fraction of sentences using passive voice |

| Sentence length variance | Standard deviation of word counts per sentence |

| Average sentence length | Mean words per sentence |

| Long sentence ratio | Fraction of sentences over 30 words |

| Short sentence ratio | Fraction of sentences under 8 words (good for AEO) |

| Pronoun-to-noun ratio | Vagueness indicator — high ratio = ambiguous references |

| Entity density | Proper noun density — measures how named/specific the content is |

| Answer-first ratio | Fraction of sentences that open with a direct answer pattern |

| Coreference leak count | Number of ambiguous pronoun references across sentence boundaries |

| Nested clause depth | Average subordinate conjunction depth per sentence |

| Abstraction ratio | Abstract nouns as fraction of all nouns |

| Q&A density | Question+answer pairs per paragraph |

| Definition density | Sentences containing "is defined as", "refers to", "means" etc. |

| List item ratio | List items as fraction of total lines |

| Heading density | Headings per 100 words |

These produce hard numeric anchors. A passive voice ratio of 0.40 is a fact — not a judgement.

**\*\*Stage 2 — Gemini scores with enforced constraints\*\***

Those 15 measurements are formatted into a grounding block that is prepended to the Gemini prompt as hard scoring instructions. Example:

...

Passive Voice Ratio: 0.40 → semanticClarity penalty: -8 pts  
Entity Density: 0.08 → entityRecognition base: 74  
Answer-First Ratio: 0.22 → aeoAlignment base: 66  
Coreference Leak Count: 3 → contentChunkingQuality penalty: -9 pts  
Heading Density: 1.2 per 100 words → logicalStructure boost: +5 pts  
...

Gemini's system instruction explicitly states: **\*\*\*Do not assign a score that contradicts these measurements.\*\*\*** Gemini cannot override the numeric anchors. It starts from them and applies its reading comprehension on top — scoring things like whether the content would actually answer a question an AI system surfaces, whether entities are named unambiguously enough to cite, whether the content matches conversational query patterns.

The result is a structured JSON object with every score field filled in. The backend then clamps every value to 0–100 and rounds to a whole number before returning it.

**\*\*Why this matters for the plugin:\*\***

The plugin must send enough real content for Stage 1 to produce meaningful measurements. If the content is too short or stripped of its structure, the algorithm has nothing to work with and scores will be artificially uniform. The plugin must never strip HTML before sending — headings, lists, and structure are measured directly.

---

## ## Additional changes required

The following changes were not in the original prompt above but are required for correct behaviour.

---

**### 8. `includes/api/class-rairo-api-client.php` — increase request timeout**

The current timeout is 30 seconds. The backend Gemini timeout is 55 seconds. If Gemini takes more than 30 seconds the plugin will show a connection error even though the backend succeeded and the analysis completed fine.

Find:

```php

```
private $timeout = 30;
...
```

Change to:

```
```php
private $timeout = 60;
...
```

---

#### 9. `includes/class-rairo-ajax.php` — send industry field from post context

The backend uses the `industry` field to contextualise Gemini's scoring. A medical article and an ecommerce product page should score differently. The plugin currently only sends `industry` if the user manually selects it in the content analyzer form. It should also try to derive it from post categories when analyzing from the post editor.

In the `analyze\_content()` method, after reading `\$industry` from `\$\_POST`, add a fallback:

```
```php
// If no industry provided, try to derive from post categories
if ( empty( $industry ) && $post_id > 0 ) {
    $categories = get_the_category( $post_id );
    if ( ! empty( $categories ) ) {
        $industry = $categories[0]->name;
    }
}
...
```

10. `includes/class-rairo-ajax.php` — enforce minimum content length before sending to backend

The readability algorithm needs sufficient text to produce meaningful scores. Analyzing a stub post produces technically valid but meaningless numbers and wastes an analysis credit.

In `analyze\_content()`, after building `\$full\_content`, add:

```
```php
$word_count = str_word_count(wp_strip_all_tags($full_content));
if ($word_count < 100) {
 wp_send_json_error(array(
 'message' => sprintf(
 /* translators: %d: minimum word count */
```

```

 __('Content is too short to analyze accurately (%d words). Please add more content —
a minimum of 100 words is recommended for meaningful scores.',
'rain-os-ai-readability-optimizer'),
 $word_count
),
 'code' => 'content_too_short',
 'word_count' => $word_count,
);
}
...

```

---

### 11. `includes/api/class-rairo-api-client.php` — send raw HTML not stripped text

The content sent to the backend must include HTML structure. The local readability algorithm in the backend measures heading density from `

# `–`<h6>` tags and list item ratio from `- ` tags directly. If the plugin strips HTML before sending, those two metrics always return zero and the scores will be artificially low for well-structured content.

Confirm that `analyze\_content()` in `class-rairo-ajax.php` is sending `\$content` via `wp\_kses\_post()` not `wp\_strip\_all\_tags()`. `wp\_kses\_post()` preserves safe HTML including headings and lists. This is already correct in the current code — just do not change it when making other edits.

---

### 12. `assets/js/admin.js` — show word count warning to user

When the backend returns a `content\_too\_short` error code, show a specific helpful message rather than a generic error.

In the AJAX error handler for the analyze call, add handling for this specific code:

```

````javascript
error: function(response) {
    var message = rainOsAeo.i18n.error;
    if (response && response.responseJSON && response.responseJSON.data) {
        var data = response.responseJSON.data;
        if (data.code === 'content_too_short') {
            message = data.message || 'Content is too short to analyze. Please add more content.';
        } else if (data.message) {
            message = data.message;
        }
    }
}
}

```

```
$results.html('<div class="rain-os-error">' + message + '</div>');  
}  
...
```

Updated summary table

Area	Issue	Fix
Pillar 4 score	Never shown anywhere	Files 1–6 above
Pillar Breakdown page	Wrong table name = blank page	File 4
Sub-scores on breakdown page	Fake numbers	File 4
Recommendations	Crashes if object format	File 3
Dashboard overall score	Divides by 3 not 4	File 5
Score history average	Divides by 3 not 4	File 6
Score panel (post editor)	content_id never sent	Files 1, 2
Crawler signals	Never displayed	Files 3, 4
Request timeout	30s vs 55s backend = false errors	File 8
Industry context	Never sent from post editor	File 9
Short content	Wastes credits, misleading scores	Files 10, 12
HTML stripping	Breaks heading/list metrics	File 11 (verify only)